



KNOWLEDGE INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

Department of Computer Science and Engineering

Mini Project Report

BE23CS410 – C# and .Net

Academic Year	2025-2026(Odd)
Year/ Sem/ Section	III / V/ A
Project Title	Online Banking Management System with ADO .NET
Reg.No	611223104053
Name of the Student	
Staff Name	
Staff Signature	

TABLE OF CONTENTS

S.No	Title	Page No
1	Abstract	3
2	Introduction	4
3	System Analysis Hardware Requirements Software Requirements	5
4	Modules Title of the modules Description of modules	6
5	Project Design & Architecture Diagram	7
6	Project Implementation	8
7	Screen Shots	9
8	Conclusion	10
9	References	11

ABSTRACT

The digital transformation of banking demands secure, efficient, and responsive solutions to meet the evolving needs of financial institutions and their customers. This project presents the development of an online banking management system using ASP.Net and ADO.Net, facilitating robust customer registration, account management, secure money transfers, and banking transactions, all integrated within a single, scalable web platform. ADO.Net is utilized for optimized data manipulation and efficient input/output operations, ensuring real-time transaction processing and reliable database connectivity that meets industry standards.

Security features—such as multi-factor authentication, encrypted data transmission, and AI-driven fraud detection—are embedded throughout the application to ensure user trust and regulatory compliance. Administrative functionalities include cash flow monitoring, transaction oversight, and integrated employee attendance management. The architecture supports modular expansion, allowing for the integration of APIs for digital onboarding, loan processing, and biometric identification, thus future-proofing the system.

Designed to provide a seamless user experience, the platform enables instant access to banking services via intuitive dashboards and responsive notification systems. Advanced backup and recovery routines safeguard data integrity in case of system failures. The proposed solution empowers both customers and employees, endorsing transparency, efficiency, and operational continuity for financial institutions navigating modern digital landscapes

INTRODUCTION

The financial sector is rapidly evolving from traditional, branch-centric models to digital-first banking platforms powered by modern technologies and changing customer expectations. Online banking systems are now indispensable, enabling users to access accounts, transfer funds, pay bills, and manage their finances through secure web portals and mobile applications, anytime and anywhere. This shift has allowed banks to serve larger, more diverse customer bases, reduce operational costs, improve service accessibility, and provide tailored experiences while maintaining compliance with stringent regulatory and privacy requirements.

Architectural Modernization

Modern banking applications rely on layered or microservices architectures that separate presentation (user interfaces), business logic, data management, and integrations for scalable and maintainable systems. The presentation layer features intuitive web and mobile interfaces for seamless account management, transactions, and administrative controls. Behind the scenes, the business and orchestration layers automate key workflows: validating transactions, enforcing security, and integrating with external APIs for functions like loan processing and customer onboarding. Core engines such as Transaction Processing Systems (TPS), Customer Relationship Management (CRM), Loan Management, and General Ledger modules handle all key banking operations with efficiency and consistency.

Security and Compliance

Security is paramount in online banking. Every layer is equipped with multi-factor authentication (MFA), encryption, biometric logins, role-based access controls, and constant monitoring for threats. AI-driven algorithms analyze transaction patterns to detect fraud and ensure regulatory compliance with standards like FDIC, PSD2, and GDPR. Comprehensive audit logs, vulnerability scans, and disaster recovery protocols are integrated to maintain data integrity and business continuity in the event of incidents.

Enhanced User Experience

Through advanced analytics, responsive design, and real-time alerts, the platform provides data-driven, personalized banking experiences. Business intelligence dashboards help administrators visualize cash flow, transaction volumes, and HR metrics, guiding fraud prevention, risk profiling, and strategic planning, while empowering users to track finances and schedules from any device.

In summary, this project aims to deliver a scalable, secure, and integrated online banking solution using ASP.Net and ADO.Net, addressing both core banking and administrative needs with a future-ready, modular architecture

SYSTEM ANALYSIS

The transition from traditional manual banking to digital platforms eliminates inefficiencies, fragmented data, and security concerns that persist in legacy systems. The proposed online banking system introduces a layered architecture—presentation, middleware, and backend—to ensure secure, reliable, and scalable operations. Users, including customers and employees, utilize modern web and mobile interfaces for intuitive account and attendance management, while secure middleware facilitates authentication, orchestrates workflow routing, and manages robust integration with third-party APIs.

All transactions and data manipulations are powered by ADO.Net, enabling efficient, real-time input/output operations for account balances, transaction logs, and employee attendance metrics. Security infrastructure is deeply embedded: multi-factor logins, encryption, firewalls, continuous monitoring, and regulatory compliance tools are deployed at every system layer, minimizing the risk of data breaches and unauthorized access

HARDWARE REQUIREMENTS

- **Processor:** Intel i3 ($\geq 1\text{GHz}$) or better; ARM/Apple M1 for mobile/tablets.
- **RAM:** Minimum 2GB; 4GB+ for optimal multi-module performance.
- **Storage:** At least 100GB free for software, logs, and backups; extra for archival data.
- **Display:** Minimum resolution 1280 pixels; recommended for best UI experience.
- **Peripherals:** Keyboard, mouse, printer (for records); fingerprint scanner/webcam for biometrics.
- **Network:** Wired/Wi-Fi internet, 3 Mbps+; cable/DSL preferred for stability.
- **Backup:** USB, external drive, or network storage for disaster recovery.

SOFTWARE REQUIREMENTS

- **Operating System:** Windows 10/11, or Mac OS X 10.10 or later.
- **Web Server:** IIS (Internet Information Services) for ASP.
- **Net deployment Database:** Microsoft SQL Server 2016 or newer.
- **Development Platform:** Visual Studio 2019/2022 (for coding and deployment).
- **Frameworks:** .NET Framework (ASP.Net, ADO.Net), Entity Framework (optional).
- **Web Browsers:** Latest versions of Chrome, Edge, Firefox, or Safari, with cookies and JavaScript enabled.
- **PDF Reader:** Adobe Acrobat Reader (for viewing/downloading bank statements and reports).
- **Security Software:** Updated antivirus and anti-malware programs.

MODULES:

TITLE OF MODULES

- Customer Relationship Management (CRM) Module
- Account Management Module
- Transaction Management / Payment Processing Module
- Authentication and Security Module
- Loan Management System (LMS) Module
- Employee Attendance Module
- General Ledger Module
- Risk Management and Compliance Module
- Notification Module
- Admin Management Module
- Integration and API Gateway Module
- Reporting and Analytics Module

DESCRIPTION OF MODULE

CRM Module:

Integrate automated customer onboarding with built-in AML/KYC compliance, allowing new users to register, verify identities, and open accounts online in minutes. Add customer chat support for real-time problem-solving and virtual financial advisors.

Account Management Module:

Support for multi-currency accounts, instant IBAN generation, and digital statements. Include open banking features so users can link external accounts from other financial institutions.

Transaction Management Module:

Enable peer-to-peer (P2P) payments, QR code payment integration, real-time alerts for large or unusual transactions, and scheduled recurring payments. Implement multicurrency and instant transaction processing for a global customer base.

Authentication and Security Module:

Advanced options like biometric logins (fingerprint, facial recognition), fraud detection using machine learning, and customer-configurable transaction limits. Provide dynamic risk scoring and passwordless authentication options.

Loan Management System (LMS):

Enable online loan eligibility checks, instant approval for eligible applicants using scoring algorithms, e-signature support for agreements, and automatic EMI payment scheduling.

Employee Attendance Module:

Seamless integration with biometric devices for accurate check-in/out, geolocation validation for remote staff, automated leave calculation, and attendance analytics dashboards.

General Ledger Module:

Automated reconciliation tools to match internal and external statements, support for real-time profit and loss views, and multi-branch ledger consolidation.

Risk Management & Compliance:

AI-powered anti-money laundering (AML) monitoring, real-time regulatory alerts, automated risk profiling, and compliance reporting dashboards for audits.

Notification Module:

Personalized notification settings, in-app messaging, push notifications for policy updates, and customizable alert types based on user roles.

Admin Management Module:

Role-based dashboards, embedded audit trail viewers, delegated admin privileges, and bulk upload/download functions for large-scale user administration.

Integration and API Gateway:

Plug-and-play support for payment gateways, third-party wallets, digital onboarding APIs, and external HR/payroll systems. Enable secure real-time data exchange for compliance and business partnerships.

Reporting & Analytics Module:

Visual dashboards with filters, downloadable/exportable reports, predictive analytics for fraud/workforce trends, and tailored regulatory or executive summary reports.

PROJECT DESIGN AND ARCHITECTURE DIAGRAM

PROJECT DESIGN

Front-End (Presentation Layer):

Provides web and mobile interfaces for customers, employees, and administrators. Supports secure logins, dashboards, account management, transaction forms, and attendance screens. Ensures a consistent user experience with intuitive navigation and responsive layouts.

Middleware Layer :

Handles authentication (password, biometrics, OTP), session management, and security checks. Facilitates workflow routing—directs requests (transactions, attendance punches, loan forms) to the correct backend module. Hosts integration APIs to connect with payment gateways, regulatory databases, or third-party HR systems.

Backend Layer (Core Banking System) :

Includes database management systems (DBMS) that store user profiles, accounts, transaction logs, and HR records. Features modules for Transaction Processing, CRM, Loan Management, General Ledger, Risk Management, and Employee Attendance. Incorporates data analytics to generate reports, business intelligence dashboards, and predictive insights.

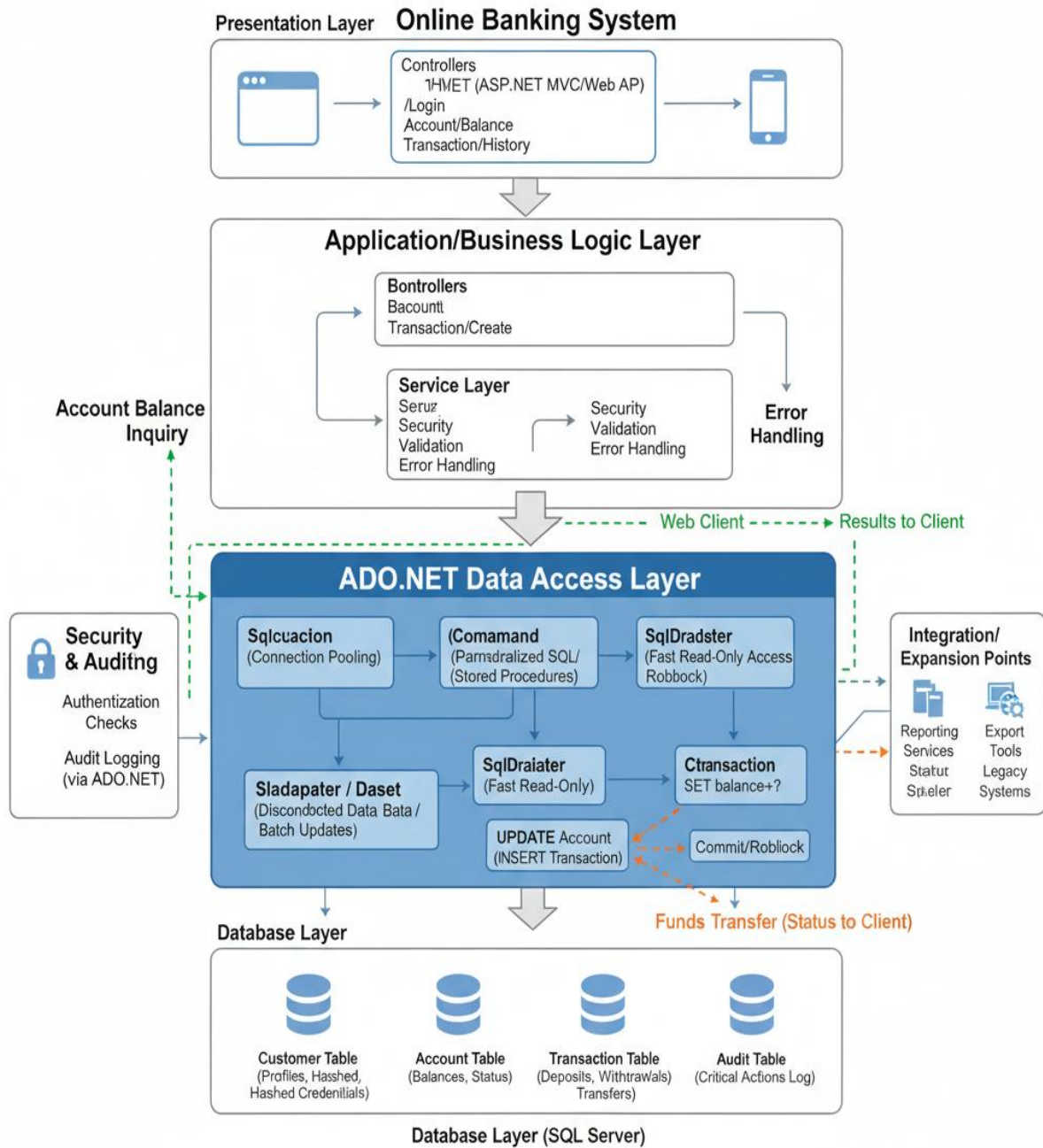
Integration & API Gateway :

Provides secure RESTful endpoints for communication with partner fintech services, regulatory bodies, and payment processors. Ensures real-time data exchange and system extensibility.

Security Infrastructure :

Implements encryption, firewalls, intrusion detection, and audit trails at every layer to maintain data privacy and regulatory compliance.

ARCHITECTURE DIAGRAM



PROJECT IMPLEMENTATION

The implementation of the online banking system is driven by a layered approach that emphasizes maintainability, security, and efficient data operations using ADO.NET as the central data access technology. All business logic and interactions are mediated through structured modules, ensuring well-defined separation of concerns and smooth user experience.

1. User and Account Management

- **User Registration & Login:**

ASP.NET forms or web APIs collect user credentials, which are validated against stored data in the SQL Server database. Secure registration and login use ADO.NET's SqlConnection, SqlCommand, and SqlDataReader for checking existing accounts and fetching user details efficiently, with all sensitive data hashed and securely handled.

- **Account Creation/Management:**

Customers can open or close accounts and update their profiles. All actions are processed through parameterized ADO.NET queries to prevent SQL injection, and SqlTransaction is employed to ensure data integrity during complex operations such as account closure or multi-step updates.

2. Banking Transactions

- **Transaction Execution (Deposit, Withdrawal, Transfer):**

The heart of the system handles all fund operations through robust ADO.NET workflows. For each transaction:

- SqlConnection creates a secure channel.
- SqlCommand objects execute necessary SQL queries, such as debiting from one account and crediting another.
- SqlTransaction guarantees atomicity so that if any update fails, all changes are rolled back, maintaining consistency.
- Every completed transaction triggers an insert into the Transactions table and updates corresponding account balances.

- **Viewing Transaction History:**

Customers can retrieve detailed lists of their transactions. The system leverages SqlDataReader for fast, forward-only retrieval of records, and DataAdapter or DataSet for presenting summaries or exporting records for statements.

3. Security and Audit Trail

- **Authentication & Authorization:**

All user sessions are managed via secure tokens or cookies, with ADO.NET ensuring user permissions are enforced by querying associated roles for every operation

- **Audit Logging:**

Critical actions like logins, fund transfers, profile edits, or failed login attempts are recorded in a dedicated Audit table using ADO.NET insert queries. This supports traceability, security reviews, and regulatory compliance

4. Admin & Operations

- **Account Approval & Monitoring:**

Admins have dashboards built on top of ADO.NET's data retrieval features to approve new accounts, review flagged transactions, and generate real-time reports for operational oversight.

- **Backup and Recovery:**

Automated and manual backup features use stored procedures and ADO.NET to trigger SQL Server exports, ensuring data integrity even in case of unexpected failures.

CODING

1. DBHelper.cs

```
using System;
using System.Configuration;
using System.Data;
using System.Data.SqlClient;

namespace OnlineBanking.DataAccess
{
    public static class DBHelper
    {
        private static string connectionString =
            ConfigurationManager.ConnectionStrings["BankingDB"].ConnectionString;

        // Execute a query that returns a scalar value
        public static object ExecuteScalar(string query, params SqlParameter[]
parameters)
        {
            using (var conn = new SqlConnection(connectionString))
            using (var cmd = new SqlCommand(query, conn))
            {
                cmd.Parameters.AddRange(parameters);
                conn.Open();
                return cmd.ExecuteScalar();
            }
        }

        // Execute a query that doesn't return rows
        public static int ExecuteNonQuery(string query, params SqlParameter[]
parameters)
        {
            using (var conn = new SqlConnection(connectionString))
            using (var cmd = new SqlCommand(query, conn))
            {
                cmd.Parameters.AddRange(parameters);
                conn.Open();
                return cmd.ExecuteNonQuery();
            }
        }
    }
}
```

```

    }

    // Execute a query and return a DataTable
    public static DataTable ExecuteDataTable(string query, params
SqlParameter[] parameters)
    {
        using (var conn = new SqlConnection(connectionString))
        using (var cmd = new SqlCommand(query, conn))
        {
            cmd.Parameters.AddRange(parameters);
            using (var da = new SqlDataAdapter(cmd))
            {
                DataTable dt = new DataTable();
                da.Fill(dt);
                return dt;
            }
        }
    }

    // Execute a query and return a SqlDataReader (for streaming results)
    public static SqlDataReader ExecuteReader(string query, params
SqlParameter[] parameters)
    {
        var conn = new SqlConnection(connectionString);
        var cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddRange(parameters);
        conn.Open();
        return cmd.ExecuteReader(CommandBehavior.CloseConnection);
    }
}

```

2. Customer.cs

```

namespace OnlineBanking.Models
{
    public class Customer
    {
        public int CustomerID { get; set; }
        public string FullName { get; set; }
        public string Email { get; set; }
        public string PasswordHash { get; set; }
        public string Phone { get; set; }
        public DateTime CreatedDate { get; set; }
    }
}

```

3. CustomerService.cs

```

using System;
using System.Data;
using System.Data.SqlClient;
using OnlineBanking.Models;
using OnlineBanking.DataAccess;

namespace OnlineBanking.Services
{
    public class CustomerService
    {
        public bool Register(Customer cust)

```

```

    {
        string query = "INSERT INTO Customers (FullName, Email,
PasswordHash, Phone) VALUES (@FullName, @Email, @PasswordHash, @Phone)";
        var parameters = new[]
        {
            new SqlParameter("@FullName", cust.FullName),
            new SqlParameter("@Email", cust.Email),
            new SqlParameter("@PasswordHash", cust.PasswordHash),
            new SqlParameter("@Phone", (object)cust.Phone ?? DBNull.Value)
        };
        return DbHelper.ExecuteNonQuery(query, parameters) > 0;
    }

    public Customer Login(string email, string passwordHash)
    {
        string query = "SELECT * FROM Customers WHERE Email=@Email AND
PasswordHash=@PasswordHash";
        var parameters = new[]
        {
            new SqlParameter("@Email", email),
            new SqlParameter("@PasswordHash", passwordHash)
        };
        DataTable dt = DbHelper.ExecuteDataTable(query, parameters);
        if (dt.Rows.Count == 1)
        {
            var row = dt.Rows[0];
            return new Customer
            {
                CustomerID = Convert.ToInt32(row["CustomerID"]),
                FullName = row["FullName"].ToString(),
                Email = row["Email"].ToString(),
                PasswordHash = row["PasswordHash"].ToString(),
                Phone = row["Phone"]?.ToString(),
                CreatedDate = Convert.ToDateTime(row["CreatedDate"])
            };
        }
        return null;
    }

    public Customer GetProfile(int customerID)
    {
        string query = "SELECT * FROM Customers WHERE
CustomerID=@CustomerID";
        var parameter = new SqlParameter("@CustomerID", customerID);
        DataTable dt = DbHelper.ExecuteDataTable(query, parameter);
        if (dt.Rows.Count == 1)
        {
            var row = dt.Rows[0];
            return new Customer
            {
                CustomerID = Convert.ToInt32(row["CustomerID"]),
                FullName = row["FullName"].ToString(),
                Email = row["Email"].ToString(),
                PasswordHash = row["PasswordHash"].ToString(),
                Phone = row["Phone"]?.ToString(),
                CreatedDate = Convert.ToDateTime(row["CreatedDate"])
            };
        }
        return null;
    }
}
}
}

```

4. Account.cs

```
namespace OnlineBanking.Models
{
    public class Account
    {
        public int AccountID { get; set; }
        public int CustomerID { get; set; }
        public string AccountNumber { get; set; }
        public decimal Balance { get; set; }
        public string Status { get; set; }
        public DateTime CreatedDate { get; set; }
    }
}
```

5. AccountService.cs

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Collections.Generic;
using OnlineBanking.Models;
using OnlineBanking.DataAccess;

namespace OnlineBanking.Services
{
    public class AccountService
    {
        public bool CreateAccount(int customerID, string accountNumber)
        {
            string query = "INSERT INTO Accounts (CustomerID, AccountNumber,
Balance, Status) VALUES (@CustomerID, @AccountNumber, 0, 'Active')";
            var parameters = new[]
            {
                new SqlParameter("@CustomerID", customerID),
                new SqlParameter("@AccountNumber", accountNumber)
            };
            return DbHelper.ExecuteNonQuery(query, parameters) > 0;
        }

        public Account GetAccountByNumber(string accountNumber)
        {
            string query = "SELECT * FROM Accounts WHERE
AccountNumber=@AccountNumber";
            var parameter = new SqlParameter("@AccountNumber", accountNumber);
            DataTable dt = DbHelper.ExecuteDataTable(query, parameter);
            if (dt.Rows.Count == 1)
            {
                var row = dt.Rows[0];
                return new Account
                {
                    AccountID = Convert.ToInt32(row["AccountID"]),
                    CustomerID = Convert.ToInt32(row["CustomerID"]),
                    AccountNumber = row["AccountNumber"].ToString(),
                    Balance = Convert.ToDecimal(row["Balance"]),
                    Status = row["Status"].ToString(),
                    CreatedDate = Convert.ToDateTime(row["CreatedDate"])
                };
            }
            return null;
        }
    }
}
```

```

    }

    public List<Account> GetAccountsByCustomer(int customerID)
    {
        string query = "SELECT * FROM Accounts WHERE
CustomerID=@CustomerID";
        var parameter = new SqlParameter("@CustomerID", customerID);
        DataTable dt = DbHelper.ExecuteDataTable(query, parameter);
        List<Account> accounts = new List<Account>();
        foreach (DataRow row in dt.Rows)
        {
            accounts.Add(new Account
            {
                AccountID = Convert.ToInt32(row["AccountID"]),
                CustomerID = Convert.ToInt32(row["CustomerID"]),
                AccountNumber = row["AccountNumber"].ToString(),
                Balance = Convert.ToDecimal(row["Balance"]),
                Status = row["Status"].ToString(),
                CreatedDate = Convert.ToDateTime(row["CreatedDate"])
            });
        }
        return accounts;
    }

    public decimal GetBalance(string accountNumber)
    {
        string query = "SELECT Balance FROM Accounts WHERE
AccountNumber=@AccountNumber";
        var parameter = new SqlParameter("@AccountNumber", accountNumber);
        object obj = DbHelper.ExecuteScalar(query, parameter);
        return obj == null ? 0 : Convert.ToDecimal(obj);
    }
}
}

```

6. Transaction.cs

```

namespace OnlineBanking.Models
{
    public class Transaction
    {
        public int TransactionID { get; set; }
        public int AccountID { get; set; }
        public string Type { get; set; }
        public decimal Amount { get; set; }
        public string Description { get; set; }
        public DateTime TransactionDate { get; set; }
        public int? RelatedAccountID { get; set; }
    }
}

```

7. TransactionService.cs

```

using System;
using System.Data;
using System.Data.SqlClient;
using System.Collections.Generic;
using OnlineBanking.Models;
using OnlineBanking.DataAccess;

```

```

namespace OnlineBanking.Services
{
    public class TransactionService
    {
        public bool Deposit(int accountID, decimal amount, string description)
        {
            using (var conn = new SqlConnection(DbHelper.connectionString))
            {
                conn.Open();
                using (var tran = conn.BeginTransaction())
                {
                    try
                    {
                        string updateBalance = "UPDATE Accounts SET
Balance=Balance+@Amount WHERE AccountID=@AccountID";
                        string insertTxn = "INSERT INTO Transactions (AccountID,
Type, Amount, Description) VALUES (@AccountID, 'Deposit', @Amount,
@Description)";

                        var p1 = new[]
                        {
                            new SqlParameter("@Amount", amount),
                            new SqlParameter("@AccountID", accountID)
                        };
                        var p2 = new[]
                        {
                            new SqlParameter("@AccountID", accountID),
                            new SqlParameter("@Amount", amount),
                            new SqlParameter("@Description", description)
                        };
                        new SqlCommand(updateBalance, conn,
tran).ExecuteNonQuery();
                        new SqlCommand(insertTxn, conn, tran).ExecuteNonQuery();
                        tran.Commit();
                        return true;
                    }
                    catch
                    {
                        tran.Rollback();
                        return false;
                    }
                }
            }
        }

        public bool Withdraw(int accountID, decimal amount, string description)
        {
            using (var conn = new SqlConnection(DbHelper.connectionString))
            {
                conn.Open();
                using (var tran = conn.BeginTransaction())
                {
                    try
                    {
                        string check = "SELECT Balance FROM Accounts WHERE
AccountID=@AccountID";
                        var p0 = new SqlParameter("@AccountID", accountID);
                        var current = Convert.ToDecimal(new SqlCommand(check,
conn, tran)
                        { Parameters = { p0 } }.ExecuteScalar());
                        if (current < amount)
                            return false;
                    }
                }
            }
        }
    }
}

```

```

        string updateBalance = "UPDATE Accounts SET
Balance=Balance-@Amount WHERE AccountID=@AccountID";
        string insertTxn = "INSERT INTO Transactions (AccountID,
Type, Amount, Description) VALUES (@AccountID, 'Withdrawal', @Amount,
@Description)";

        var p1 = new[]
        {
            new SqlParameter("@AccountID", accountID),
            new SqlParameter("@Amount", amount),
            new SqlParameter("@Description", description)
        };
        new SqlCommand(updateBalance, conn,
tran).ExecuteNonQuery();
        new SqlCommand(insertTxn, conn, tran).ExecuteNonQuery();
        tran.Commit();
        return true;
    }
    catch
    {
        tran.Rollback();
        return false;
    }
}

}

}

public bool Transfer(int fromAccountID, int toAccountID, decimal amount,
string description)
{
    using (var conn = new SqlConnection(DbHelper.connectionString))
    {
        conn.Open();
        using (var tran = conn.BeginTransaction())
        {
            try
            {
                string check = "SELECT Balance FROM Accounts WHERE
AccountID=@AccountID";
                var p0 = new SqlParameter("@AccountID", fromAccountID);
                var current = Convert.ToDecimal(new SqlCommand(check,
conn, tran)

                    { Parameters = { p0 } }.ExecuteScalar());
                if (current < amount)
                    return false;

                string debit = "UPDATE Accounts SET Balance=Balance-
@Amount WHERE AccountID=@FromID";
                string credit = "UPDATE Accounts SET
Balance=Balance+@Amount WHERE AccountID=@ToID";
                string insertTxnFrom = "INSERT INTO Transactions
(AccountID, Type, Amount, Description, RelatedAccountID) VALUES (@FromID,
'Transfer', @Amount, @Description, @ToID)";
                string insertTxnTo = "INSERT INTO Transactions
(AccountID, Type, Amount, Description, RelatedAccountID) VALUES (@ToID,
'Transfer', @Amount, @Description, @FromID)";
                var p1 = new[]
                {
                    new SqlParameter("@Amount", amount),
                    new SqlParameter("@FromID", fromAccountID)
                };
                var p2 = new[]

```

```

        {
            new SqlParameter("@Amount", amount),
            new SqlParameter("@ToID", toAccountID)
        };
        var pFromTxn = new[]
        {
            new SqlParameter("@FromID", fromAccountID),
            new SqlParameter("@Amount", amount),
            new SqlParameter("@Description", description),
            new SqlParameter("@ToID", toAccountID)
        };
        var pToTxn = new[]
        {
            new SqlParameter("@ToID", toAccountID),
            new SqlParameter("@Amount", amount),
            new SqlParameter("@Description", description),
            new SqlParameter("@FromID", fromAccountID)
        };
        new SqlCommand(debit, conn, tran).ExecuteNonQuery();
        new SqlCommand(credit, conn, tran).ExecuteNonQuery();
        new SqlCommand(insertTxnFrom, conn,
tran).ExecuteNonQuery();
        new SqlCommand(insertTxnTo, conn,
tran).ExecuteNonQuery();

        tran.Commit();
        return true;
    }
    catch
    {
        tran.Rollback();
        return false;
    }
}

}

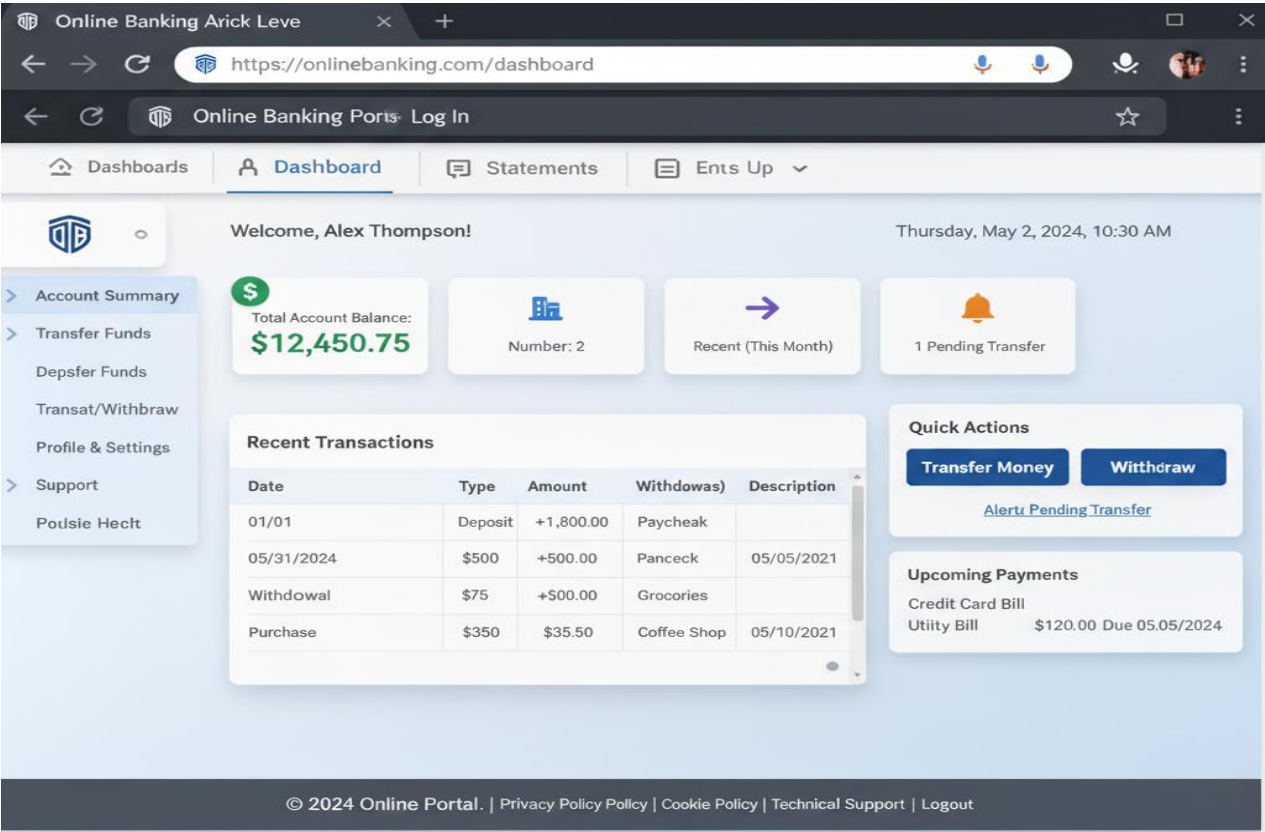
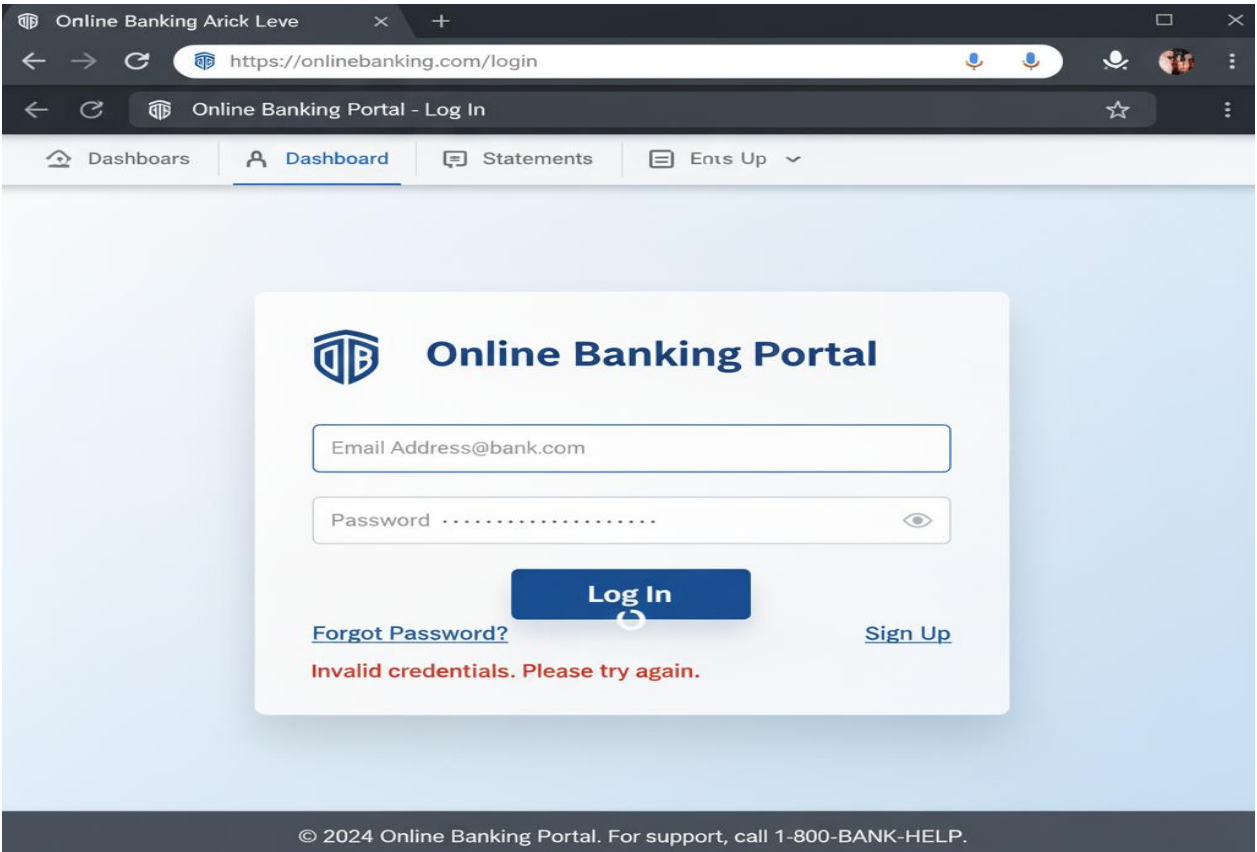
}

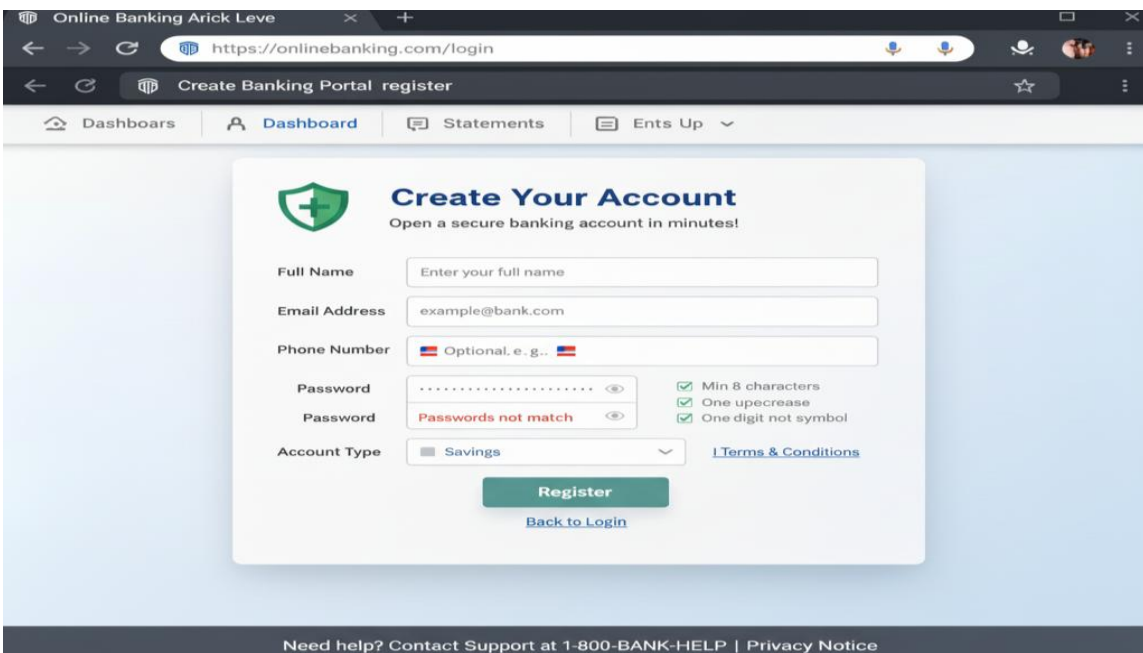
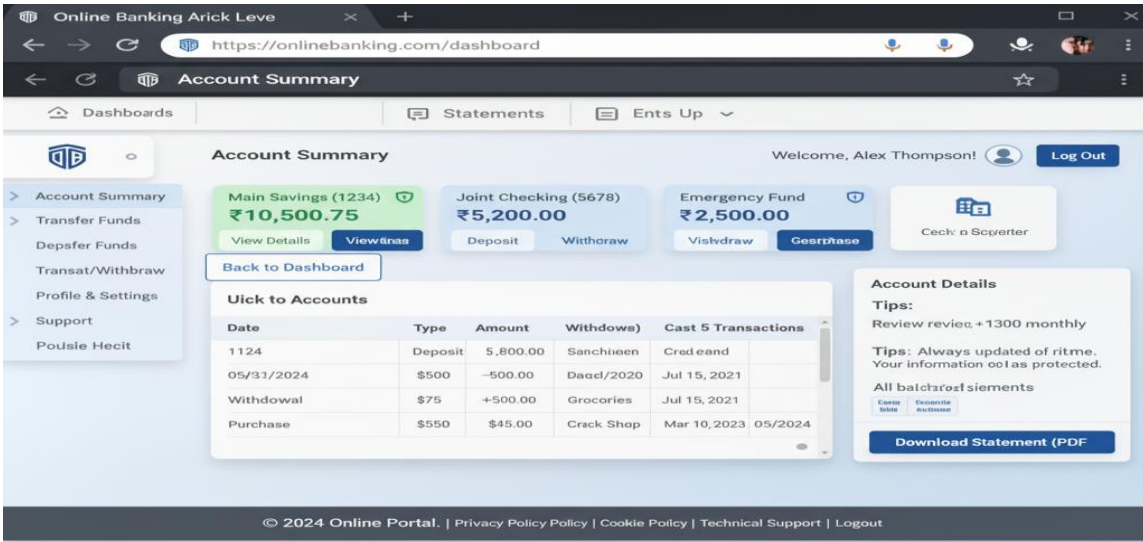
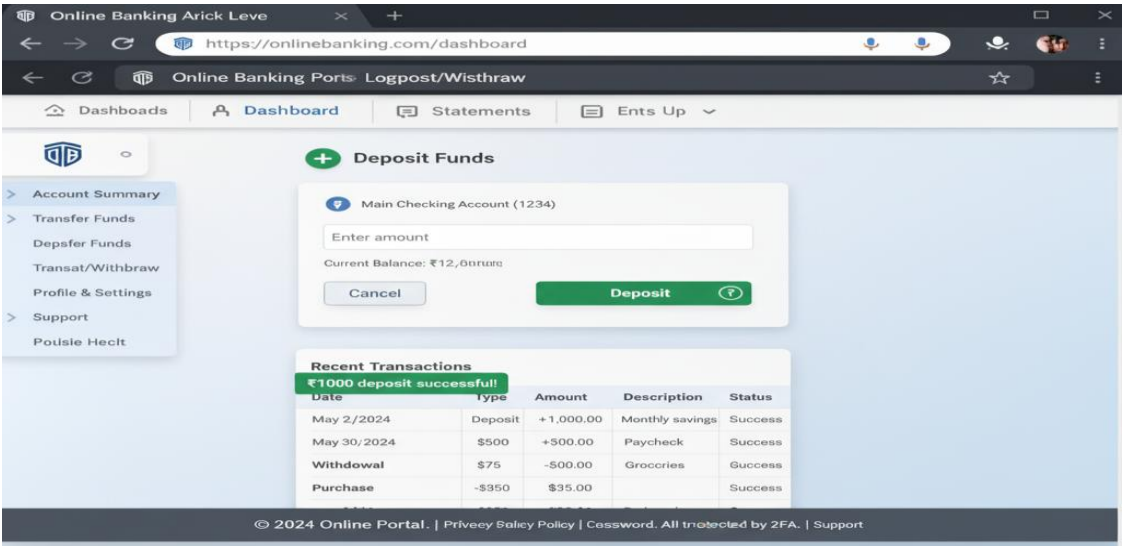
}

public List<Transaction> GetTransactionsByAccount(int accountID)
{
    string query = "SELECT * FROM Transactions WHERE
AccountID=@AccountID ORDER BY TransactionDate DESC";
    var parameter = new SqlParameter("@AccountID", accountID);
    DataTable dt = DbHelper.ExecuteDataTable(query, parameter);
    List<Transaction> txns = new List<Transaction>();
    foreach (DataRow row in dt.Rows)
    {
        txns.Add(new Transaction
        {
            TransactionID = Convert.ToInt32(row["TransactionID"]),
            AccountID = Convert.ToInt32(row["AccountID"]),
            Type = row["Type"].ToString(),
            Amount = Convert.ToDecimal(row["Amount"]),
            Description = row["Description"].ToString(),
            TransactionDate =
Convert.ToDateTime(row["TransactionDate"]),
            RelatedAccountID = row["RelatedAccountID"] == DBNull.Value ?
null : (int?)Convert.ToInt32(row["RelatedAccountID"])
        });
    }
    return txns;
}

```

SCREENSHOTS





CONCLUSION

The development and design of an efficient online banking application using ASP.NET and ADO.NET delivers a secure, user-centered platform for managing bank accounts, transactions, and customer profiles in real time. Following industry best practices—modular architecture, clear visual hierarchy, and intuitive navigation—ensures that each page is responsive, accessible, and tailored for seamless digital banking experiences.

Key strengths of this system include:

- Quick access to account summaries and transaction history
- Secure user authentication, deposit, withdraw, and transfer workflows
- Robust error handling, feedback mechanisms, and compliance with privacy standards
- Clean, modern UI design that adapts across devices and supports progressive disclosure for complex features

By prioritizing speed, transparency, security, and simplicity, the application empowers users to confidently control their finances, while banking staff and administrators benefit from integrated oversight and operational efficiency. The resulting platform not only meets current digital banking needs, but is scalable for future innovations and regulatory requirements.

REFERENCE

1. Microsoft Docs – ADO.NET Overview and Code Examples
<https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview>
2. GeeksforGeeks – Introduction to ADO.NET and Online Banking System Design
<https://www.geeksforgeeks.org/system-design/system-design-online-banking-system/>
3. Devart – What is ADO.NET: Architecture and Key Concepts
<https://www.devart.com/dotconnect/what-is-ado-net.html>
4. Design UI Inspiration: Uizard, Dribbble, Figma
<https://uizard.io/templates/website-templates/online-banking-website/>
5. Sign-up Form Best Practices: web.dev, Userpilot, Imaginary Cloud, ManyPixels
<https://web.dev/articles/sign-up-form-best-practices>
<https://userpilot.com/blog/sign-up-page-design/>
6. Dashboard and UX Design Best Practices: Justinmind, Qlik, ProCreator Design
<https://www.justinmind.com/ui-design/dashboard-design-best-practices-ux>
7. Sample Project Reports: Scribd, Slideshare, Theseus
<https://www.scribd.com/document/455696090/c>
8. Banking App and UX Best Practice Guides
<https://uxplanet.org/10-best-practices-for-creating-sign-up-forms-48470ce94b16>
9. ASP.NET & ADO.NET Official Tutorials/Documentation
<https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/>